调度、驱逐、抢占、压制

核心问题 用户配置的资源 匹配到了高优先级qos 如何解决

先来整理下相关逻辑

qos

1. Guaranteed：Pod 中的所有容器都定义了相同的资源请求和限制，且请求等于限制。
2. Burstable：Pod 中的容器至少定义了资源请求，但请求和限制不相等。
3. BestEffort：Pod 中的容器没有定义资源请求或限制。

在调度时，调度器优先考虑具有更高 QoS 类别的 Pod，以确保系统的资源分配和稳定性。

核心几个过程：

过程中有几个过程考虑：

1. 调度决策：在选择合适的节点来调度 Pod 时，调度器会考虑 pod谁来优先调度。（kube-scheduler）
2. 抢占策略：在调度filter到score阶段 无法调度的时候。会出发抢占逻辑。尝试删除更低优先级的pod 来满足调度要求 （kube-scheduler）
3. 驱逐策略：在节点资源紧张时，kubelet会根据驱逐触发的场景，来选择排序pod进行驱逐，以保障其他容器持续运行。（kubelet）
4. 杀死策略：分为容器内oom和全局oom，超过cgroup限制时会触发。会调用select_bad_process函数 进行杀死(OOM)

调度

调度队列 按优先级排在调度队列前面 schedulingQueue当中 在schedulerOne方法中 直接NextPod pop一个元素出来

队列的排序方法 主要是看优先级

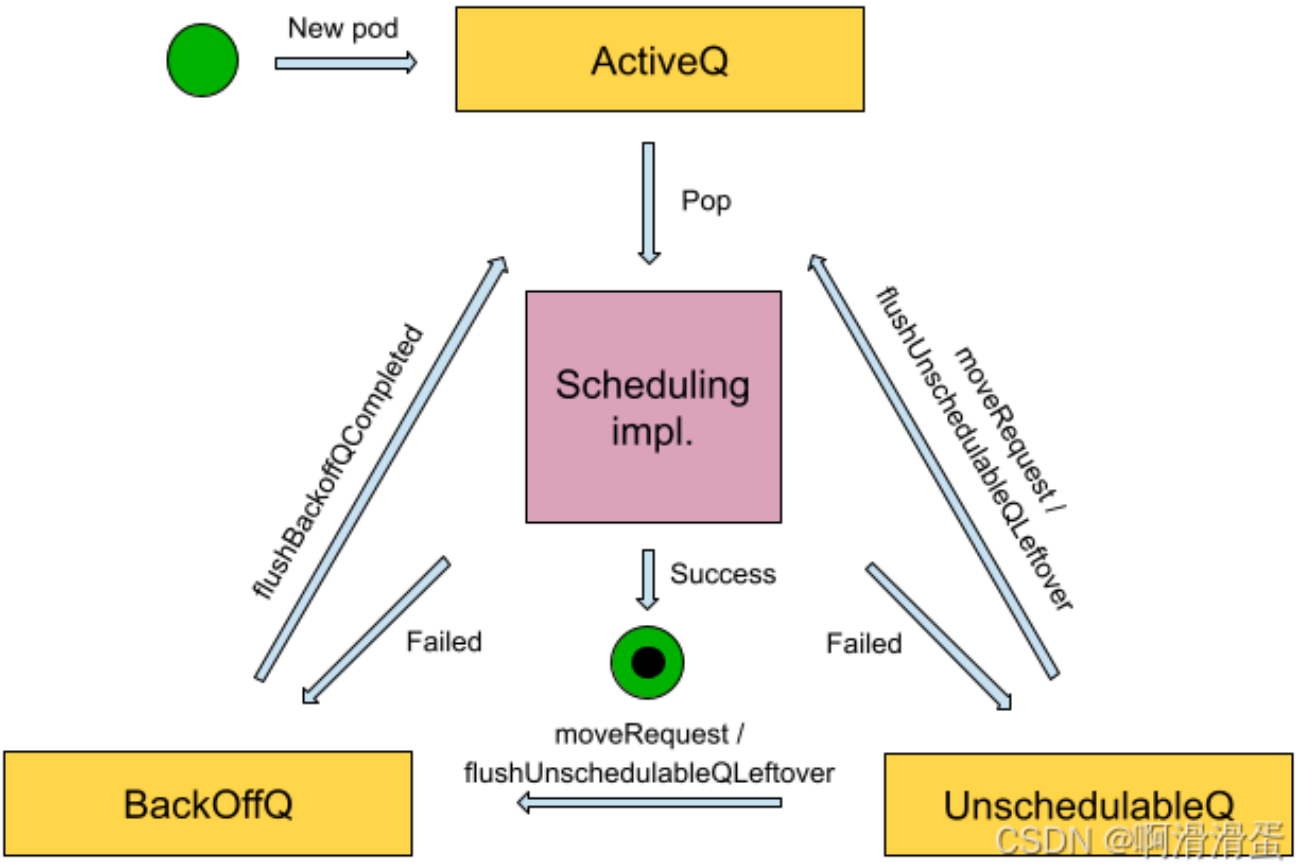
默认情况下priorityClass优先级都是0 -2,147,483,648 到 1,000,000,000

新家的pod都是加入activeQ当中，activeQ在PriorityQueue当中

priorityQueue头部就是优先级最高的待处理pod

里面有几个队列

- activeQ 正在考虑调度的pod
- backoffq 从unschedulablepod移动的pod backoff结束后移到activeQ
- unschedulabelPod 当前明确不可调度的pod



Pod 优先级对调度顺序的影响

当启用 Pod 优先级时，调度程序会按优先级对悬决 Pod 进行排序， 并且每个悬决的 Pod 会被放置在调度队列中其他优先级较低的悬决 Pod 之前。 因此，如果满足调度要求，较高优先级的 Pod 可能会比具有较低优先级的 Pod 更早调度。 如果无法调度此类 Pod，调度程序将继续并尝试调度其他较低优先级的 Pod。

源码

```
// PriorityQueue implements a scheduling queue.
// The head of PriorityQueue is the highest priority pending pod. This structure
// has two sub queues and a additional data structure, namely: activeQ,
// backoffQ and unschedulablePods.
//   - activeQ holds pods that are being considered for scheduling.
//   - backoffQ holds pods that moved from unschedulablePods and will move to
//     activeQ when their backoff periods complete.
//   - unschedulablePods holds pods that were already attempted for scheduling and
//     are currently determined to be unschedulable.
type PriorityQueue struct {  86 usages
```

activeQ是一个heap queue实现的heap

```
// activeQ is heap structure that scheduler actively looks at to find pods to
// schedule. Head of heap is the highest priority pod.
activeQ *heap.Heap
```

排序逻辑初始化时候注入

```
podQueue := internalqueue.NewSchedulingQueue(
    profiles[options.profiles[0].SchedulerName].QueueSortFunc(),
    informerFactory,
    internalqueue.WithPodInitialBackoffDuration(time.Duration(options.podInitialBackoffSeconds)*time.Second),
    internalqueue.WithPodMaxBackoffDuration(time.Duration(options.podMaxBackoffSeconds)*time.Second),
    internalqueue.WithPodLister(podLister),
    internalqueue.WithPodMaxInUnschedulablePodsDuration(options.podMaxInUnschedulablePodsDuration),
    internalqueue.WithPreEnqueuePluginMap(preEnqueuePluginMap),
    internalqueue.WithQueueingHintMapPerProfile(queueingHintsPerProfile),
    internalqueue.WithPluginMetricsSamplePercent(pluginMetricsSamplePercent),
    internalqueue.WithMetricsRecorder(*metricsRecorder),
)
```

```
podQueue := internalqueue.NewSchedulingQueue(
    profiles[options.profiles[0].SchedulerName].QueueSortFunc(),
    informerFactory,
    internalqueue.WithPodInitialBackoffDuration(time.Duration(options.podInitialBackoffSeconds)*time.Second),
    internalqueue.WithPodMaxBackoffDuration(time.Duration(options.podMaxBackoffSeconds)*time.Second),
    internalqueue.WithPodLister(podLister),
    internalqueue.WithPodMaxInUnschedulablePodsDuration(options.podMaxInUnschedulablePodsDuration),
    internalqueue.WithPreEnqueuePluginMap(preEnqueuePluginMap),
    internalqueue.WithQueueingHintMapPerProfile(queueingHintsPerProfile),
    internalqueue.WithPluginMetricsSamplePercent(pluginMetricsSamplePercent),
    internalqueue.WithMetricsRecorder(*metricsRecorder),
)
```

```go
// Less is the function used by the activeQ heap algorithm to sort pods.
// It sorts pods based on their priority. When priorities are equal, it uses
// PodQueueInfo.timestamp.
func (pl *PrioritySort) Less(pInfo1, pInfo2 *framework.QueuedPodInfo) bool {
    p1 := corev1helpers.PodPriority(pInfo1.Pod)
    p2 := corev1helpers.PodPriority(pInfo2.Pod)
    return (p1 > p2) || (p1 == p2 && pInfo1.Timestamp.Before(pInfo2.Timestamp))
}
```

Less是activeQ堆算法用来对pod进行排序的函数。它根据 pod 的优先级对 pod 进行排序。当优先级相等时，它使用PodQueueInfo.timestamp。

思路

需要增加PriorityClass供在离线pod使用

两种策略

- 在线和离线均配置。在线的更高 （优势：未来可扩展更多的QOS）． 匹配型。 做扩展 需要工作分类规则和匹配规则
- 离线配置为负，比任何pod的都低 （前期选择）

驱逐

- 资源压力的时候 （kubelet主动驱逐）
- 调度的时候（抢占）

主动驱逐Pod

自我修复行为

会先尝试回收资源 由垃圾回收死亡的pod和容器 删除未使用的镜像

驱逐信号

Linux 系统中，kubelet 使用以下驱逐信号：

驱逐信号	描述
`memory.available`	`memory.available := node.status.capacity[memory] - node.stats.memory.workingSet`
`nodefs.available`	`nodefs.available := node.stats.fs.available`
`nodefs.inodesFree`	`nodefs.inodesFree := node.stats.fs.inodesFree`
`imagefs.available`	`imagefs.available := node.stats.runtime.imagefs.available`
`imagefs.inodesFree`	`imagefs.inodesFree := node.stats.runtime.imagefs.inodesFree`
`pid.available`	`pid.available := node.stats.rlimit.maxpid - node.stats.rlimit.curproc`

在上表中，**描述**列显示了 kubelet 如何获取信号的值。每个信号支持百分比值
或者是字面值。 kubelet 计算相对于与信号有关的总量的百分比值。

来自cgroupfs

可用内存 memory.avilable

节点的根文件系统或镜像文件系统上的可用磁盘空间和 inode

nodefs 主机主要的文件系统 本地磁盘卷 emptyDir卷 日志存储等 如/var/lib/kubelet

imagefs 容器镜像和可写层

驱逐条件

软驱逐 设置的驱逐 不会立即驱逐

硬驱逐 立即驱逐 会直接杀死pod

kubelet 具有以下默认硬驱逐条件：

- `memory.available<100Mi`
- `nodefs.available<10%`
- `imagefs.available<15%`
- `nodefs.inodesFree<5%` （Linux 节点）
- `imagefs.inodesFree<5%` (Linux 节点)

监控间隔

驱逐监测间隔

kubelet 根据其配置的 `housekeeping-interval` （默认为 `10s` ）评估驱逐条件。

需要先于原生驱逐触发 如何避免同时触发驱逐 定义水位线 警戒线

可以配置一些软驱逐等条件和宽限期

驱逐pod时的选择

回收资源使得驱逐信号低于条件 如果做不到的话 驱逐

驱逐的顺序

kubelet 使用以下参数来确定 Pod 驱逐顺序：



1. Pod 的资源使用是否超过其请求
2. Pod 优先级
3. Pod 相对于请求的资源使用情况

因此，kubelet 按以下顺序排列和驱逐 Pod：

1. 首先考虑资源使用量超过其请求的 `BestEffort` 或 `Burstable` Pod。这些 Pod 会根据它们的优先级以及它们的资源使用级别超过其请求的程度被逐出。
2. 资源使用量少于请求量的 `Guaranteed` Pod 和 `Burstable` Pod 根据其优先级被最后驱逐。

> **说明：**
> kubelet 不使用 Pod 的 QoS 类来确定驱逐顺序。 在回收内存等资源时，你可以使用 QoS 类来估计最可能的 Pod 驱逐顺序。 QoS 分类不适用于临时存储（EphemeralStorage）请求， 因此如果节点在 `DiskPressure` 下，则上述场景将不适用。

驱逐进程和inode

当 kubelet 因 inode 或 进程 ID 不足而驱逐 Pod 时， 它使用 Pod 的相对优先级来确定驱逐顺序，因为 inode 和 PID 没有对应的请求字段。

nodefs的使用情况（本地卷+所有容器的日志）进行排序 imagefs会根据容器的可写层排序

最小回收量

可以配置回收直到配置值

oom杀死 不同于驱逐

节点内存不足 非驱逐 oom_killer接管

如果 kubelet 在节点遇到 OOM 之前无法回收内存， 则 oom_killer 根据它在节点上使用的内存百分比计算 oom_score， 然后加上 oom_score_adj 得到每个容器有效的 oom_score。 然后它会杀死得分最高的容器。

这意味着低 QoS Pod 中相对于其调度请求消耗内存较多的容器，将首先被杀死。

与 Pod 驱逐不同，如果容器被 OOM 杀死， kubelet 可以根据其 restartPolicy 重新启动它。

源码

kubelet中 关键调用链路evictionManager Synchronize evictPod

数据来源于cadvisor 有先后依赖关系 我们自己实现 也需要cadvisor的数据

```
    }
    // eviction manager must start after cadvisor because it needs to know
    kl.evictionManager.Start(kl.StatsProvider, kl.GetActivePods, kl.PodIsF
```

cadvisor获取的节点内存使用数据

pod的排序也跟驱逐场景相关

```
    // rank the running pods for eviction for the specified resource
    rank(activePods, statsFunc)
```

```
// rankMemoryPressure orders the input pods for eviction in response to memory pressure.
// It ranks by whether or not the pod's usage exceeds its requests, then by priority, and
// finally by memory usage above requests.
func rankMemoryPressure(pods []*v1.Pod, stats statsFunc) {  3 usages
    orderedBy(exceedMemoryRequests(stats), priority, memory(stats)).Sort(pods)
}

// rankPIDPressure orders the input pods by priority in response to PID pressure.
func rankPIDPressure(pods []*v1.Pod, stats statsFunc) {  2 usages
    orderedBy(priority, process(stats)).Sort(pods)
}

// rankDiskPressureFunc returns a rankFunc that measures the specified fs stats.
func rankDiskPressureFunc(fsStatsToMeasure []fsStatsType, diskResource v1.ResourceName) rankFunc {  13 usages
    return func(pods []*v1.Pod, stats statsFunc) {
        orderedBy(exceedDiskRequests(stats, fsStatsToMeasure, diskResource), priority, disk(stats, fsStatsToMeasure, diskResource)).Sort(pods)
    }
}
```

```
// priority compares pods by Priority, if priority is enabled.
func priority(p1, p2 *v1.Pod) int {  12 usages
    priority1 := corev1helpers.PodPriority(p1)
    priority2 := corev1helpers.PodPriority(p2)
    if priority1 == priority2 : 0
    if priority1 > priority2 : 1
    return -1
}
```

以memoryPressure为例

1. exceedMemoryRequests 使用量超出request需求
2. 优先级高低
3. 内存使用的大小

实现了sort接口

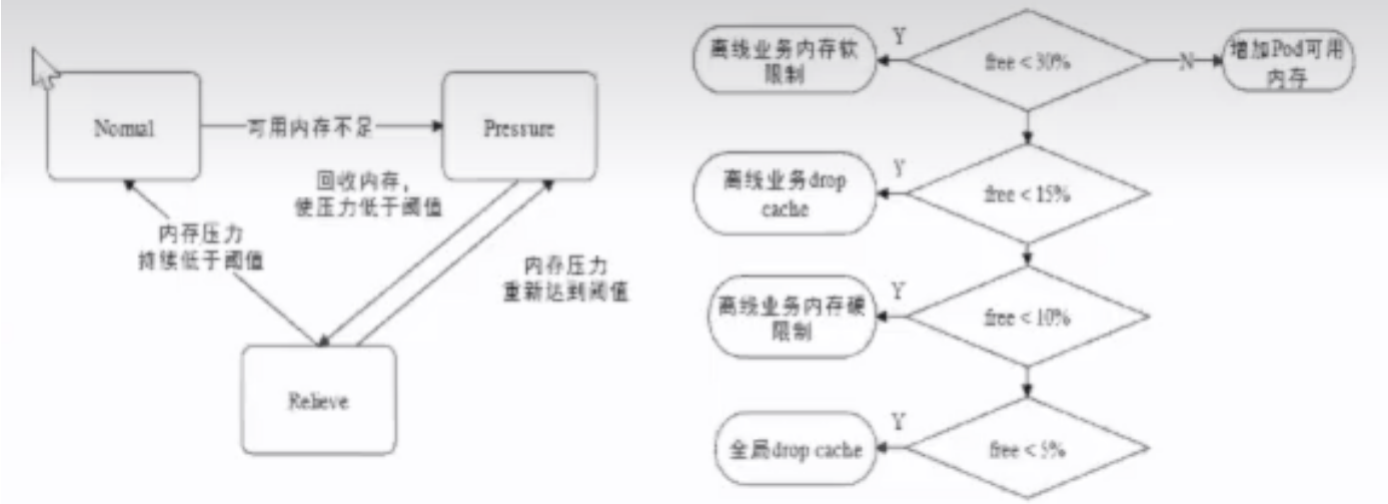
驱逐不按qos类驱逐　按照优先级来驱逐 所以和我们的优先级设计息息相关。 那么第一个其实是qos的隐形规则 扩展资源不受影响

> **说明：**
> kubelet 不使用 Pod 的 QoS 类来确定驱逐顺序。 在回收内存等资源时，你可以使用 QoS 类来估计最可能的 Pod 驱逐顺序。 QoS 分类不适用于临时存储（EphemeralStorage）请求， 因此如果节点在 `DiskPressure` 下，则上述场景将不适用。

思路

需要在agent实现驱逐策略 抢在原生驱逐前

rubik当前提供了mem分级回收　动态回收 调整离线业务的memory cgroup 主动释放内存的page cache



<30% 释放限制离线任务使用的内存 不会kill 主动drop 离线业务的cache

<10% 主动硬限制 进行oom

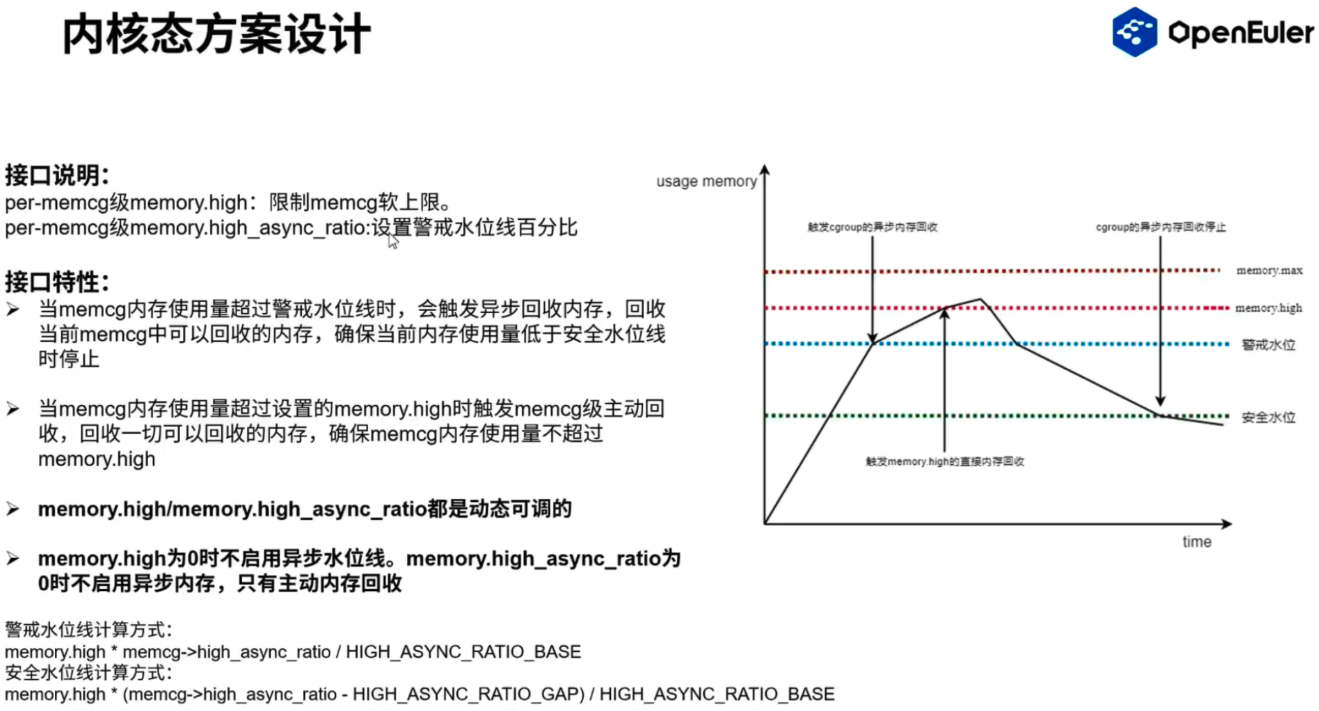
<5% 无法保证在线业务 全局drop cache

接口：

配置键/文件名[=默认值]	描述	可选值	归属
memoryConfig	内存分级回收相关配置	0,1	rubik
.enable=false	是否使能memoryConfig功能	true,false	
.strategy=none	内存动态分级回收控制策略	none, dynlevel,	
.checkInterval=5	周期性检查的时间	1-30	
volcano.sh/preemptable="false"	true：离线业务，false：在线业务	"false","true"	用户

周期是秒 **和原生间隔一致** 10s

警戒线和水位线 由rubik异步水位线功能支持



rubik不涉及驱逐 是否需要单独拿出来实现。 优势：优化在离线的资源使用和控制

目前不实现的话 原生可以保证 在线业务正常运行和驱逐离线排序在先。（选择）

抢占

无法调度的时候 会触发抢占和驱逐 低优先级pod

在执行 filter 和 score 的所有"检查点"后， 如果没有找到合适的 node，触发抢占

配置非抢占的话 preemptionPolicy: Never ， 会在调度队列低优先级pod之前，留在调度队列中，等待直到有足够大可用资源（非抢占可以被其他高优先级pod抢占）

Pod 被创建后会进入队列等待调度。 调度器从队列中挑选一个 Pod 并尝试将它调度到某个节点上。 如果没有找到满足 Pod 的所指定的所有要求的节点，则触发对悬决 Pod 的抢占逻辑。 让我们将悬决 Pod 称为 P。抢占逻辑试图找到一个节点， 在该节点中删除一个或多个优先级低于 P 的 Pod，则可以将 P 调度到该节点上。 如果找到这样的节点，一个或多个优先级较低的 Pod 会被从节点中驱逐。 被驱逐的 Pod 消失后，P 可以被调度到该节点上。

抢占逻辑：

- 驱逐受害pod
- 拒绝watingPod
- 清理低优先级pod（抢占核心）

调度器的抢占逻辑在选择抢占目标时**不考虑 QoS**。 抢占会考虑 **Pod 优先级**并尝试选择一组优先级最低的目标

PriorityClass -2,147,483,648 到 1,000,000,000

仅当移除优先级最低的 Pod 不足以让调度程序调度抢占式 Pod， 或者最低优先级的 Pod 受 PodDisruptionBudget 保护时，才会考虑优先级较高的 Pod。（移除驱逐过程中才会考虑qos kubelet驱逐顺序）

源码

preemption/preemption.go

Candidate 调度的抢占节点和受害者列表

func Preempt 返回一个节点Name和postFilterResult 以及一个status

不同场景语义不同

几种返回 即使抢占也不可调度 抢占成功 等等

selectCandidate选择合适的候选者并且返回 主要是选择Node 得分函数OrderedScoreFuncs

prepareCandidate 准备工作

- 驱逐受害pod
- 拒绝watingPod
- 清理低优先级pod（抢占核心）

```
preemptPod := func(index int) {
    victim := c.Victims().Pods[index]
    // If the victim is a WaitingPod, send a reject message to the PermitPlugin.
    // Otherwise we should delete the victim.
    if waitingPod := fh.GetWaitingPod(victim.UID); waitingPod != nil {
        waitingPod.Reject(pluginName, msg: "preempted")
        logger.V( level: 2).Info( msg: "Preemptor pod rejected a waiting pod", keysAndValues...: "preemptor", klog.KObj(pod), "waitingPod", klog.KObj(victim), "node", c.Na
    } else {
        if feature.DefaultFeatureGate.Enabled(features.PodDisruptionConditions) {
            condition := &v1.PodCondition{
                Type:    v1.DisruptionTarget,
                Status:  v1.ConditionTrue,
                Reason:  v1.PodReasonPreemptionByScheduler,
                Message: fmt.Sprintf( format: "%s: preempting to accommodate a higher priority pod", pod.Spec.SchedulerName),
            }
            newStatus := pod.Status.DeepCopy()
            updated := apipod.UpdatePodCondition(newStatus, condition)
            if updated {
                if err := util.PatchPodStatus(ctx, cs, victim, newStatus); err != nil {
                    logger.Error(err, msg: "Could not add DisruptionTarget condition due to preemption", keysAndValues...: "pod", klog.KObj(victim), "preemptor", klog.
                    errCh.SendErrorWithCancel(err, cancel)
                    return
                }
            }
        }
        if err := util.DeletePod(ctx, cs, victim); err != nil {
            logger.Error(err, msg: "Preempted pod", keysAndValues...: "pod", klog.KObj(victim), "preemptor", klog.KObj(pod))
            errCh.SendErrorWithCancel(err, cancel)
            return
        }
        logger.V( level: 2).Info( msg: "Preemptor Pod preempted victim Pod", keysAndValues...: "preemptor", klog.KObj(pod), "victim", klog.KObj(victim), "node", c.Name())
    }
```

拒绝pod 否则deletePod

```go
// Lower priority pods nominated to run on this node, may no longer fit on
// this node. So, we should remove their nomination. Removing their
// nomination updates these pods and moves them to the active queue. It
// lets scheduler find another place for them.
nominatedPods := getLowerPriorityNominatedPods(logger, fh, pod, c.Name())
if err := util.ClearNominatedNodeName(ctx, cs, nominatedPods...); err != nil {
    logger.Error(err, msg: "Cannot clear 'NominatedNodeName' field")
    // We do not return as this error is not critical.
}

return nil
```

指明不适合这个节点 会重新加入active queue 调度到新的地方

```go
func getLowerPriorityNominatedPods(logger klog.Logger, pn framework.PodNominator, pod *v1.Pod, nodeName string) []*v1.Pod {  2 usages
    podInfos := pn.NominatedPodsForNode(nodeName)

    if len(podInfos) == 0 { return nil }

    var lowerPriorityPods []*v1.Pod
    podPriority := corev1helpers.PodPriority(pod)
    for _, pi := range podInfos {
        if corev1helpers.PodPriority(pi.Pod) < podPriority {
            lowerPriorityPods = append(lowerPriorityPods, pi.Pod)
        }
    }
    return lowerPriorityPods
}
```

小于当前pod优先级

这里注意 期望每个节点很少的pod提名

思路

目前的抢占逻辑 结合优先级的设置 可以满足要求

离线优先级 负数

压制 绝对压制

Rubik 绝对抢占 memory_qos_enable 内存qos 当前支持memory 抢占压制

在线任务注释 volcano.sh/preemptable=false cpu qos隔离特性 实现对在离线任务的压制 cpu绝对的抢占效果

关键特性一：Pod CPU/MEM QoS配置

加入SIG群，添加小助手微信WildLittleHelper，邀您

OpenEuler

痛点：

Linux原生组调度方案提供了对不同分组任务配置CPU/MEM配额的功能，但即使在极端配置下，依然无法限制住离线任务偶发运行，从而干扰在线任务的相应时延。

场景：

离线任务在运行时，支持在线任务快速抢占离线任务及在线任务能够压制离线任务获得CPU/MEM运行。

接口：

配置键/文件名[=默认值]	描述	可选值	归属
memcg_qos_enable=0	是否使能内存qos功能	0,1	内核
volcano.sh/preemptable="false"	true：离线业务 false：在线业务	"false","true"	用户

用法：

步骤1：使能memory qos

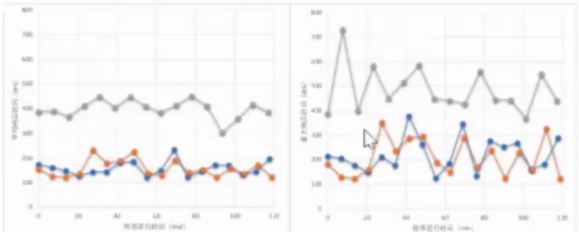
```
echo 1 > /proc/sys/vm/memcg_qos_enable
```

步骤2：配置pod的yaml文件中的注解volcano.sh/preemptable

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: qosexample
  annotations:
    volcano.sh/preemptable: "true"
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
        cpu: "1"
      requests:
        memory: "200Mi"
        cpu: "1"
```

效果：

CloudSuite验证CPU隔离效果，使用web-serving模拟在线业务，in-memory-analytics模拟离线业务，开启CPU QoS隔离特性，在离线混合部署，在线业务QoS不受影响。

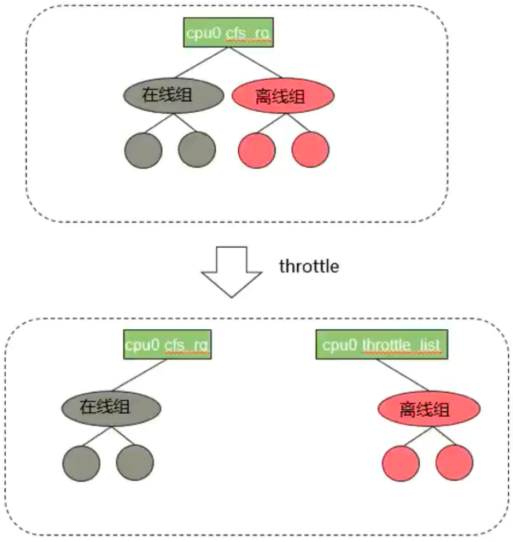


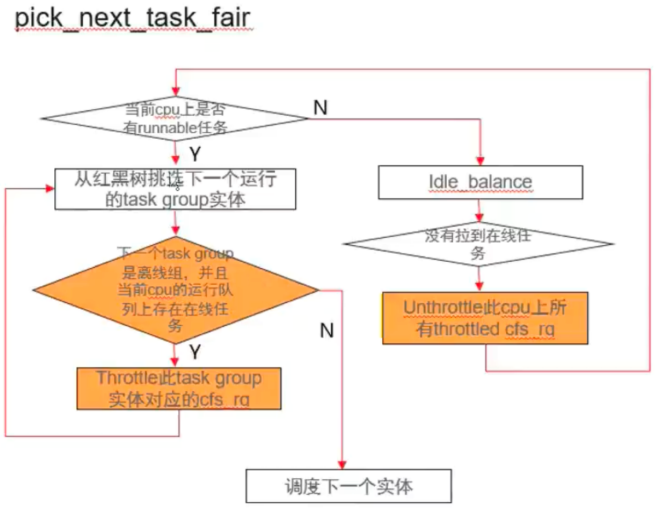
在线绝对压制离线 此外还有一些压制特性

方案设计---Qos throttle

OpenEuler

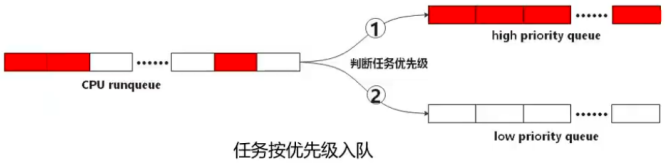
实现思路： 将离线组对应的cfs_rq 添加到一个全局percpu链表throttle_list，从而避免对在线任务产生干扰



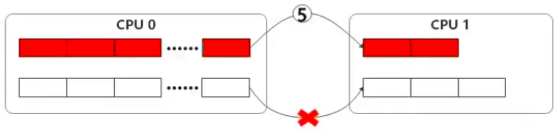


负载均衡 迁移一些任务

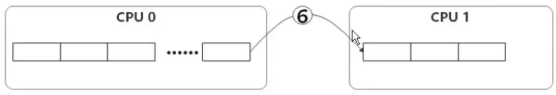
方案设计---优先级负载均衡



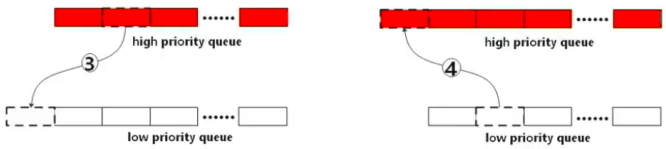
任务按优先级入队



负载均衡——高优先级任务迁移



负载均衡——低优先级任务迁移



高低优先级任务切换

1. 调度器判断任务优先级：**若为高优先级，任务入队到高优先级队列**。
2. 调度器判断任务优先级：**若为低优先级，任务入队到低优先级队列**。
3. 降低任务优先级时，高优先级任务切换到低优先级队列。
4. 提升任务优先级时，低优先级任务切换到高优先级队列。
5. **有高优先级任务运行时**，负载均衡从CPU 0上的**高优先级队列**迁移任务到CPU 1，**低优先级任务不迁移**。
6. **没有高优先级任务运行时**，负载均衡从CPU 0上的**低优先级队列**迁移任务到CPU 1。

离线任务较多 迁移有限制

不同的队列 在线任务 多核

提供了在线任务 切换成离线任务队列 修改指针 但是目前用不到

Cpu空闲的时候 迁移在线过去 离线迁移还要做抢占检测 如果没有在线任务 进行离线任务的迁移

方案设计---优先级负载均衡

优先级负载均衡具体流程：

1、调度器中断触发不同核之间的任务负载均衡；

2、分别计算本端CPU及远端CPU的任务负载；

3、如果本端CPU与远端CPU之间负载不均衡，执行负责均衡；否则退出流程；

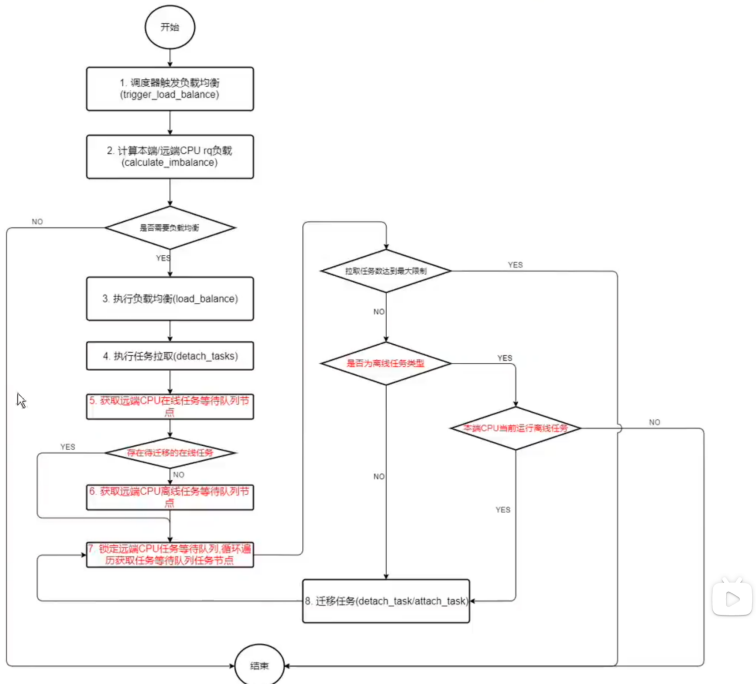
4、负载均衡在本端CPU上拉取远端CPU任务等待队列中的任务；

5、优先获取远端CPU在线任务等待队列节点，若在线任务等待队列中存在可迁移任务，跳到步骤7；

6、获取远端CPU离线任务等待队列中的任务；

7、锁定远端CPU任务等待队列，循环遍历获取任务等待队列任务；

8、执行迁移任务之前判断当前待迁移任务类型，若待迁移任务类型为在线任务，执行任务迁移，迁移后压制本端CPU离线任务调度；若待迁移任务类型为离线任务，需判断本端CPU任务运行状态：
- 当前运行在线任务，避免冗余check preempt，放弃迁移，负载均衡流程结束
- 当前运行离线任务，说明当前CPU为空闲核，可以迁移，保证离线任务不饿死



任务放到等待队列 优先级进行迁移

现有cfs_loadbalance的优化

干扰

需求介绍

OpenEuler

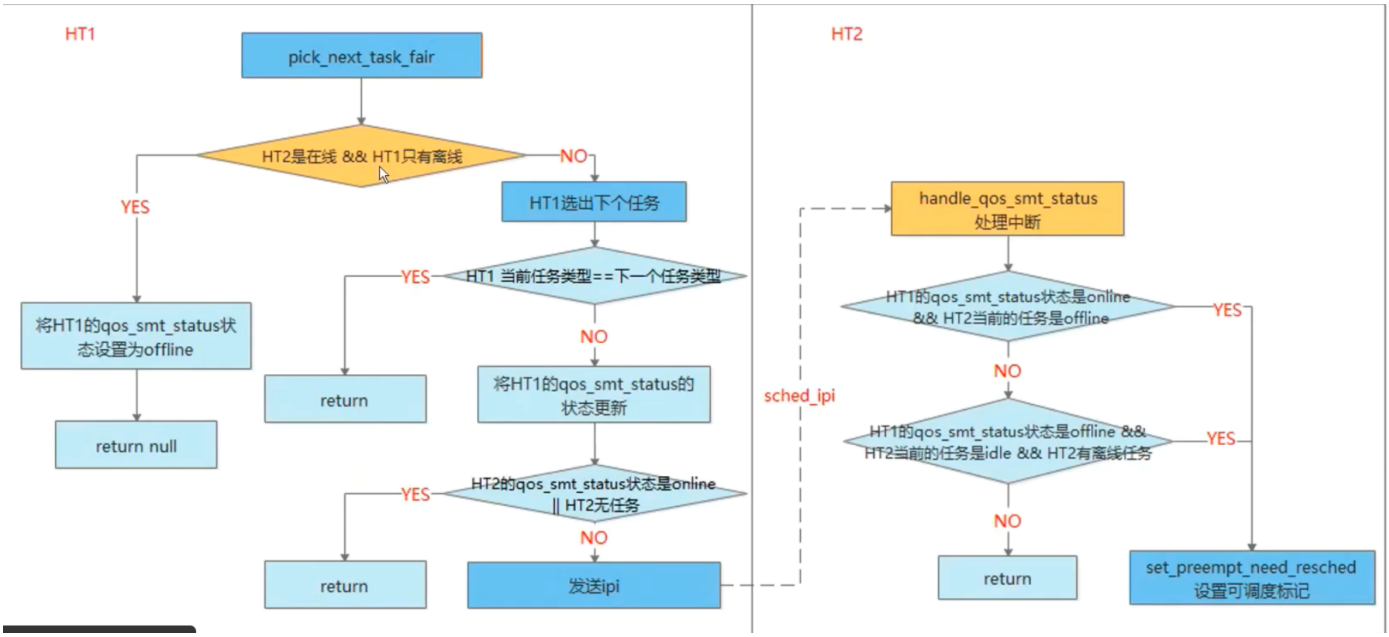
背景

同一个物理 CPU 的超线程共享核心的硬件资源，如 Cache 和计算单元。当在线任务和离线任务同时运行在一对超线程上时，会因为硬件资源争抢出现相互干扰的情况

解决方案

当在线任务和离线任务同时运行在一对超线程上时，驱离离线任务，该HT核进入idle





ipi中断到核

驱离离线任务

OOM

服务质量	oom_score_adj
Guaranteed	-997
BestEffort	1000
Burstable	**min(max(2, 1000 - (1000 * memoryRequestBytes) / machineMemoryCapacityBytes), 999)**

和rubik结合起来　增加了优先级

rubik当中是把OOM优先级划分的 优先杀死离线任务

oom_score_adj打分因子和内存使用情况评分　和原生一致

两个接口

- /proc/sys/vm/memcg_qos_enable 打开Memcg OOM分级功能
- memory.qos_level -1表示低优先级 0表示高优先级

rubik 全局oom

定义了两个memory cg 低优先级里面再选 在kill流程之前检查低优先级的memcg

memory.usage_in_bytes的memory cg kill里面task最高的进程

rubik 容器内oom

超过限制 也是杀死进程　超过cgroup中限制 杀死容器内进程

out_of_memory函数 这是无法回收内存 需要杀死的时候 才会触发

select_bad_process (发生oom的时候 cgroup级别的oom 全局) 选了一个useageMax memory cg

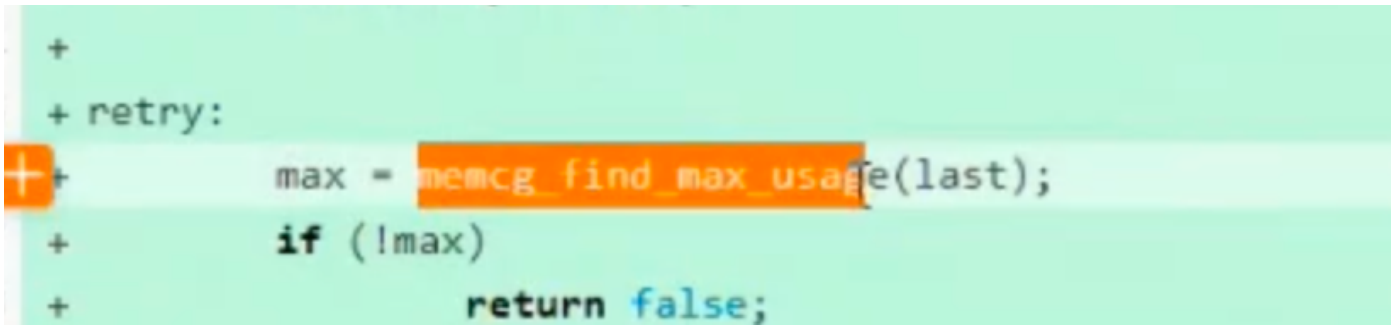
rubik的细节 我就不细说了



```
 */
static void select_bad_process(struct oom_control *oc)
{
        oc->chosen_points = LONG_MIN;

        if (is_memcg_oom(oc))
                mem_cgroup_scan_tasks(oc->memcg, oom_evaluate_task, oc);
        else {
                struct task_struct *p;

+#ifdef CONFIG_MEMCG_QOS
+               if (memcg_low_priority_scan_tasks(oom_evaluate_task, oc))
+                       return;
+#endif
                rcu_read_lock();
                for_each_process(p)
                        if (oom_evaluate_task(p, oc))
```



```
+
+ retry:
+         max = memcg_find_max_usage(last);
+         if (!max)
+                 return false;
```

重调度

调度器处

感知资源使用。均匀资源使用率。主要是混部扩展资源 （后期） 扩展资源

控制顺序

驱逐 压制禁止调度

压制rubik自身

压制驱逐 rubik原生分级回收+k8s原生驱逐

禁止调度 oom Rubik+oom。 禁止调度

实现

agent驱逐需要实现 触发要在kubelet驱逐之前 通过水位控制 要有足够的预留 不会冲突

当前rubik提供了mem动态回收 以及memcg异步水位线 cpu提供了绝对抢占qos压制 需要结合

调度器混部优先级、混部抢占（调度测）需要实现

调度感知重编排需要实现 避免单节点资源过高